

**МИНИСТЕРСТВО НАУКИ, ВЫСШЕГО ОБРАЗОВАНИЯ И ИННОВАЦИЙ
КЫРГЫЗСКОЙ РЕСПУБЛИКИ
КЫРГЫЗСКО-ГЕРМАНСКИЙ ИНСТИТУТ ПРИКЛАДНОЙ ИНФОРМАТИКИ**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

на тему: «Использование блокчейн-технологий для построения системы
электронных голосований»

Студент группы

(подпись)

Бактыбеков Нурсултан

Бактыбекович

Научный

руководитель

(подпись)

к.т.н., доцент, Самсалиев А.А.

Бишкек – 2026

АННОТАЦИЯ

Бул бүтүрүүчү квалификациялык иште электрондук добуш берүүдө айкындуулукту, коопсуздукту жана маалыматтардын бүтүндүгүн камсыз кылуу маселеси каралат. Изилдөө объектиси болуп электрондук добуш берүү системасы саналат. Иштин максаты – жогорку ылдамдыктагы Solana блокчейн тармагында смарт-контракттарга негизделген борборлоштурулбаган добуш берүү системасын иштеп чыгуу. Иштин жыйынтыгында борборлоштурулбаган тиркеменин (DApp) архитектурасы иштелип чыгып, Anchor фреймворкун колдонуу менен Rust тилинде смарт-контракт, React негизинде колдонуучунун веб-тиркемеси жана маалыматтарды топтоо үчүн Go тилинде сервердик бөлүк (блок сканери) жазылды. Иштелип чыккан система эки жолу добуш берүүдөн криптографиялык коргоону (PDA механизмин колдонуу менен), жыйынтыктардын өзгөрбөстүгүн жана жыйынтык чыгаруу процессинин ачыктыгын камсыз кылат.

В данной выпускной квалификационной работе рассматривается проблема обеспечения прозрачности, безопасности и целостности данных при проведении электронных голосований. Объектом исследования является система электронного голосования. Целью работы является разработка и реализация децентрализованной системы голосования на базе смарт-контрактов в высокоскоростной блокчейн-сети Solana. В результате выполнения работы была спроектирована архитектура децентрализованного приложения (DApp), написан смарт-контракт на языке Rust с использованием фреймворка Anchor, разработано пользовательское веб-приложение на React и серверная часть (сканер блоков) на языке Go для агрегации данных. Разработанная система обеспечивает криптографическую защиту от двойного голосования (с использованием механизма PDA), неизменяемость результатов и открытость процесса подведения итогов.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 АНАЛИТИЧЕСКАЯ ЧАСТЬ	7
1.1 Исследование и анализ предметной области темы ВКР	7
1.2 Анализ моделей, методов и программных продуктов для решения подобных задач (международная и отечественная практика)	9
1.3 Постановка и формализация задачи на разработку. Техническое задание	11
2 ПРОЕКТНАЯ ЧАСТЬ	14
2.1 Разработка архитектуры и функциональных требований	14
2.2 Проектирование моделей базы данных: логической и физической . . .	16
2.3 Обоснование выбора аппаратных средств и выбранного стека технологий	19
3 ПРОГРАММНАЯ ЧАСТЬ	22
3.1 Разработка интерфейса и программного продукта	22
3.2 Результаты тестирования программного продукта	24
3.3 Руководство пользователя, программиста	25
ЗАКЛЮЧЕНИЕ	30
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	32
ПРИЛОЖЕНИЯ	35
Приложение 1 Листинг смарт-контракта системы голосования (язык Rust)	35
Приложение 2 Листинг сервиса-сканера сети (язык Go)	39
Приложение 3 Листинг пользовательского веб-интерфейса (React)	43

ВВЕДЕНИЕ

Актуальность темы исследования. В современном информационном обществе процессы принятия решений, в том числе голосования на различных уровнях (от локальных корпоративных опросов до государственных выборов), все больше смещаются в цифровую плоскость. Традиционные методы голосования с использованием бумажных бюллетеней связаны с высокими затратами на логистику, печать, привлечение наблюдателей и организацию избирательных участков. Кроме того, ручной подсчет голосов неизбежно подвержен риску человеческого фактора, ошибок и преднамеренных фальсификаций.

Развитие технологий привело к появлению систем электронного голосования (E-voting), которые решают многие из этих проблем, повышая доступность выборов и скорость подведения итогов. Однако классические системы электронного голосования, построенные на базе централизованных клиент-серверных архитектур и реляционных баз данных (таких как PostgreSQL, MySQL или Oracle), порождают новую, еще более сложную проблему — проблему «единой точки отказа» и абсолютного доверия к администратору базы данных. Любой субъект, обладающий полным доступом к центральному серверу, может незаметно изменить результаты голосования, удалить негодные голоса или добавить новые, не оставляя при этом очевидных следов для независимых наблюдателей.

Внедрение технологии распределенного реестра (блокчейн) открывает принципиально новые горизонты в решении проблемы доверия. Блокчейн по своей природе представляет собой децентрализованную, неизменяемую и прозрачную базу данных. Благодаря криптографическому связыванию блоков и механизмам консенсуса, распределенная сеть гарантирует, что транзакция (в данном случае — голос), однажды записанная в реестр, не может быть изменена или удалена задним числом. Любой независимый наблюдатель может развернуть собственный узел сети и математически проверить корректность

подсчета голосов, что обеспечивает свойство End-to-End Verifiability (сквозной проверяемости), критически важное для демократических процедур.

Несмотря на очевидные преимущества, ранние поколения блокчейн-сетей (например, Ethereum) страдали от низкой пропускной способности (около 15 транзакций в секунду) и высоких сетевых комиссий (газа), что делало их непригодными для массовых голосований. Появление современных высокопроизводительных сетей, таких как Solana, использующих инновационный алгоритм Proof-of-History (PoH), позволяет обрабатывать десятки тысяч транзакций в секунду при минимальных затратах, снимая технологические ограничения для полномасштабного внедрения блокчейна в процессы E-voting.

Цель выпускной квалификационной работы: проектирование, разработка и программная реализация децентрализованной, безопасной и проверяемой системы электронного голосования на базе высокопроизводительной блокчейн-сети Solana.

Для достижения указанной цели в работе поставлены и решены следующие **задачи**:

1. Проведение комплексного анализа предметной области, выявление уязвимостей традиционных систем голосования и изучение мирового опыта применения блокчейн-технологий в данной сфере.
2. Формализация требований к разрабатываемой системе и составление технического задания.
3. Проектирование децентрализованной архитектуры приложения (DApp), включая разработку логической и физической моделей данных в условиях ограничений аккаунтной модели Solana.
4. Обоснование выбора программно-аппаратного стека технологий.
5. Программная реализация смарт-контракта (on-chain program) для строгой обработки логики голосования и предотвращения повторных транзакций.

6. Программная реализация пользовательского веб-интерфейса (Frontend) для взаимодействия избирателей с блокчейн-сетью посредством криптовалютных кошельков.
7. Программная реализация независимого серверного приложения (блок-сканера) для непрерывного аудита смарт-контракта, агрегации данных и их визуализации.
8. Проведение комплекса тестирований (модульного и интеграционного) разработанного программного обеспечения.

Объект исследования: информационные системы электронного голосования.

Предмет исследования: методы, алгоритмы и криптографические технологии построения защищенных децентрализованных приложений на базе блокчейна.

Научная новизна и практическая значимость. Практическая значимость работы заключается в создании готового к эксплуатации прототипа децентрализованной системы голосования, которая решает проблему масштабируемости за счет использования экосистемы Solana. Разработанный комплекс (включающий смарт-контракт, DApp и независимый сервис аудита) может быть адаптирован для проведения реальных корпоративных опросов, голосований в совете директоров (DAO) или использован в качестве основы для разработки государственных систем электронного голосования нового поколения.

Структура работы. Выпускная квалификационная работа состоит из введения, трех глав, заключения, списка использованных источников и приложений. Первая глава посвящена анализу предметной области, изучению существующих аналогов и формализации требований. Во второй главе приведено проектирование системы, детальное описание архитектуры, концепции Program Derived Addresses (PDA) и моделей данных. В третьей главе изложена программная реализация каждого компонента комплекса, представлены результаты тестирования и руководства пользователей.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Исследование и анализ предметной области темы ВКР

Голосование является фундаментальным процессом в любой социальной структуре. С технологической точки зрения, эволюция систем голосования прошла путь от устных опросов и бумажных бюллетеней до электромеханических машин и современных цифровых систем. В последние десятилетия концепция электронного голосования (E-voting) получила широкое распространение во многих странах мира (Эстония, Швейцария, США, Россия и др.).

Основными требованиями, предъявляемыми к идеальной системе электронного голосования, согласно международным стандартам информационной безопасности, являются:

1. **Целостность (Integrity):** ни один голос не может быть изменен, удален или признан недействительным в процессе передачи и хранения. Подсчет должен быть математически точным.
2. **Легитимность (Eligibility):** право голоса предоставляется только авторизованным и верифицированным участникам.
3. **Уникальность (Uniqueness):** каждый участник имеет право проголосовать только один раз в рамках одного опроса.
4. **Приватность и анонимность (Privacy):** невозможность достоверно связать личность избирателя с его выбором, что защищает от политического преследования.
5. **Защита от принуждения (Coercion Resistance):** участник должен иметь возможность доказать факт принуждения или изменить свой голос, если на него оказывалось давление.
6. **Сквозная проверяемость (End-to-End Verifiability):** избиратель может убедиться, что его голос был учтен корректно («индивидуальная проверяемость»), а любой независимый аудитор может убедиться

в правильности итогового подсчета без нарушения анонимности («универсальная проверяемость»).

В традиционных централизованных ИТ-архитектурах достижение сквозной проверяемости представляет собой крайне сложную задачу. Типовая система состоит из веб-сервера, базы данных SQL и серверов аутентификации. Главный концептуальный недостаток такой модели — необходимость доверия третьему лицу (Trusted Third Party). Администратор системы, имеющий доступ уровня root, теоретически способен выполнить команду `UPDATE votes SET candidate_id = X WHERE voter_id = Y`, и выявить этот факт внешнему аудитору практически невозможно, если логи системы также контролируются администратором.

Именно поэтому технология блокчейн рассматривается криптографами и инженерами как наиболее перспективное решение проблемы электронного голосования. Блокчейн — это распределенная база данных, которая поддерживается сетью равноправных узлов. Данные в блокчейне организованы в виде цепочки блоков, где каждый последующий блок содержит криптографический хэш предыдущего.

Ключевые свойства блокчейна, решающие проблемы E-voting:

- **Децентрализация:** отсутствие единого центра управления означает, что нет единого сервера, который можно взломать (DDoS-атаки или проникновение).
- **Консенсус:** данные добавляются в сеть только после того, как большинство независимых узлов (валидаторов) подтвердят их корректность по строгим математическим правилам.
- **Неизменяемость:** чтобы изменить один голос, записанный несколько блоков назад, злоумышленнику придется пересчитать хэши всех последующих блоков и убедить более 50% узлов сети принять эту измененную версию, что вычислительно и экономически нецелесообразно.
- **Прозрачность реестра:** транзакции находятся в публичном доступе. Развернув ноду или обратившись к Block Explorer, любой гражданин

может скачать всю базу данных и самостоятельно просуммировать голоса, обеспечив тем самым универсальную проверяемость.

Для того чтобы исключить человеческий фактор в момент самого подсчета, в блокчейн-сетях используются «Смарт-контракты» (Smart Contracts). Смарт-контракт — это компьютерная программа или протокол транзакции, который автоматически исполняется при выполнении определенных условий, заложенных в его код. Код контракта загружается в блокчейн и становится таким же неизменяемым, как и данные. Таким образом, логика «Если избиратель еще не голосовал, добавить 1 к счетчику кандидата» выполняется децентрализованной сетью строго по алгоритму, и никто не может изменить этот алгоритм в процессе выборов.

1.2 Анализ моделей, методов и программных продуктов для решения подобных задач (международная и отечественная практика)

Идея проведения выборов на блокчейне не нова, и на международном рынке существует несколько заметных проектов, каждый из которых использует различные платформы и алгоритмы консенсуса.

Проект Polys (Лаборатория Касперского) Система Polys построена на базе смарт-контрактов Ethereum. Она использует передовые алгоритмы шифрования и предлагает удобный интерфейс. Однако базовая архитектура Ethereum использует алгоритм Proof-of-Stake (ранее Proof-of-Work) с глобальным ограничением размера блока, что приводит к пропускной способности не более 15-30 TPS. Для массовых голосований система обычно разворачивается в виде приватного форка Ethereum (Private Blockchain), что несколько снижает уровень децентрализации и доверия, так как ноды управляются только авторизованными участниками.

Проект Voatz (США) Американская платформа Voatz применялась на муниципальных выборах в ряде штатов. Она использует блокчейн на базе Hyperledger для хранения данных и биометрию для аутентификации. Несмотря на коммерческий успех, независимые исследователи из MIT обнаружили в

системе Voatz ряд уязвимостей, связанных с централизованным сервером, который выступал посредником между приложением и блокчейном, что подтверждает важность истинной децентрализации DApp.

Проект Follow My Vote Данный проект использует блокчейн BitShares с консенсусом Delegated Proof-of-Stake (DPoS). Он предоставляет высокий уровень масштабируемости, однако имеет высокий порог входа для пользователей и сложную токеномику, что усложняет процесс обычного голосования для неквалифицированных пользователей.

В таблице 1 приведено сравнение популярных блокчейн-платформ с точки зрения их применимости для электронного голосования.

Таблица 1 – Сравнение базовых блокчейн-платформ для смарт-контрактов

Характеристика	Bitcoin	Ethereum	Solana
Алгоритм консенсуса	PoW (Proof of Work)	PoS (Proof of Stake)	PoH + PoS
Смарт-контракты	Ограниченные (Script)	Тьюринг-полные (Solidity)	Тьюринг-полные (Rust/C)
Скорость (TPS)	7	15-30	до 65 000
Время блока	10 минут	12 секунд	400 миллисекунд
Стоимость транзакции (голоса)	Высокая (\$2-\$20)	Высокая (\$1-\$50)	Доли цента (\$0.00025)
Целесообразность для E-Voting	Низкая	Средняя (для локальных)	Высокая (для массовых)

Из проведенного анализа очевидно, что для построения масштабируемой системы голосования необходимо использовать блокчейн нового поколения. Платформа **Solana** выбрана в качестве основы для данной выпускной квалификационной работы благодаря инновационному алгоритму Proof of History (PoH), который позволяет валидаторам синхронизировать время без

ожидания подтверждения от других узлов сети. Это дает колоссальный прирост производительности, делая каждую отправку голоса практически мгновенной для пользователя и не требующей больших финансовых затрат на оплату газа (комиссий).

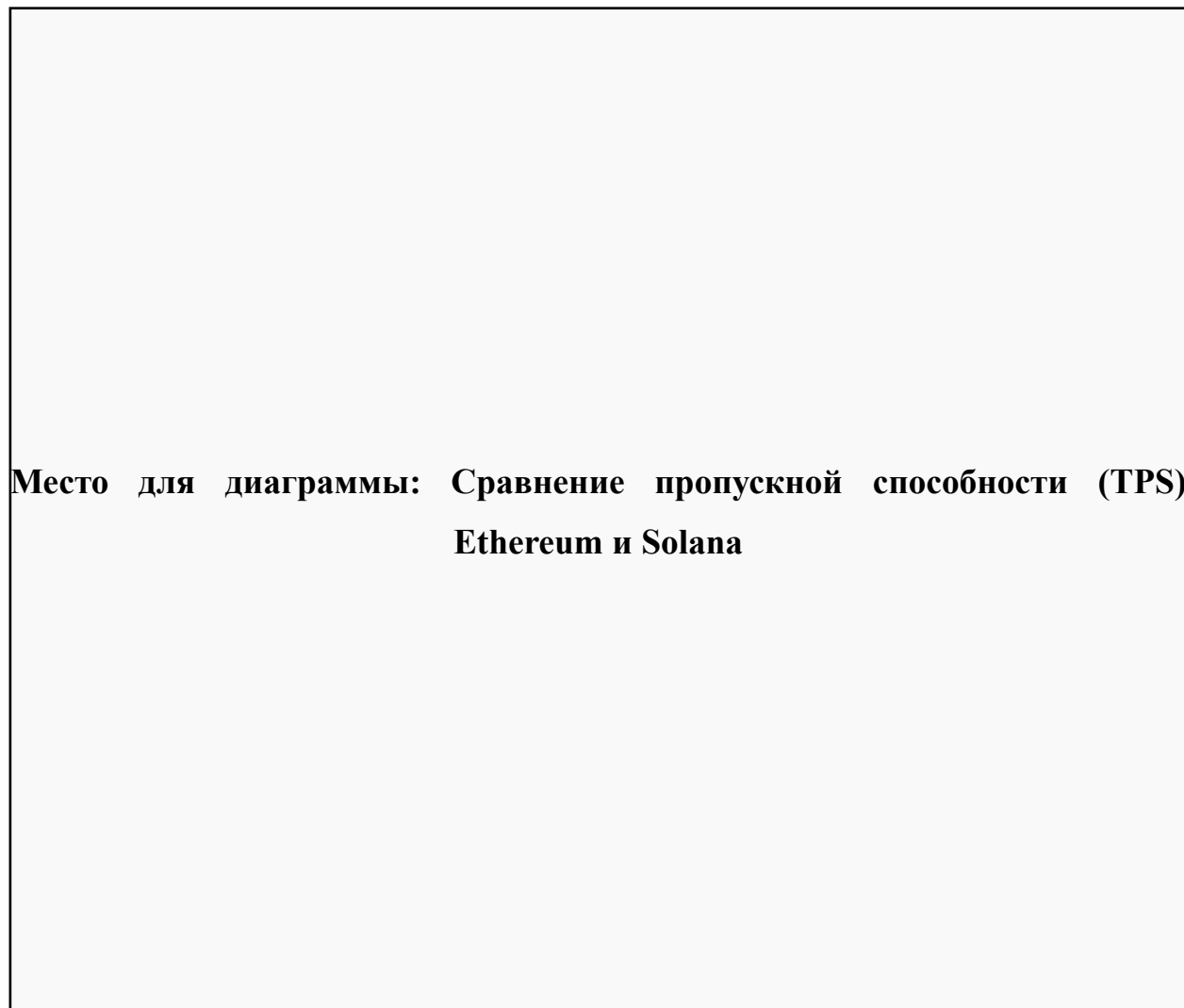


Рисунок 1 – Сравнительная диаграмма пропускной способности различных блокчейнов

1.3 Постановка и формализация задачи на разработку. Техническое задание

На основе анализа предметной области и обзора аналогов была поставлена задача: разработать программный комплекс децентрализованного электронного голосования, обеспечивающий прозрачный сбор и подсчет голосов с использованием блокчейн-сети Solana.

Техническое задание на разработку:

1. Назначение разработки: Система предназначена для проведения электронных опросов и голосований в доверенной, распределенной среде без возможности модификации результатов администраторами.

2. Требования к функциональным характеристикам:

- Подсистема аутентификации должна использовать криптографические Web3-кошельки (например, Phantom или Solflare), авторизация осуществляется путем подписи транзакции приватным ключом.
- Пользователь (инициатор) должен иметь возможность создавать новое голосование (Proposal), задавая текстовое описание (до 50 символов) и уникальный идентификатор (PID).
- Пользователи (избиратели) должны иметь возможность просматривать список активных голосований в режиме реального времени.
- Пользователь должен иметь возможность отдать голос «За» (YES) или «Против» (NO).
- Система должна блокировать на уровне смарт-контракта любые попытки проголосовать дважды за одно и то же предложение с одного кошелька.
- Должен быть предусмотрен независимый сервис-сканер, который в реальном времени мониторит блокчейн, фильтрует транзакции голосования и сохраняет их в реляционную базу данных для аудита и быстрого отображения.

3. Требования к надежности и безопасности:

- Бизнес-логика подсчета голосов должна быть реализована исключительно на стороне смарт-контракта (on-chain), чтобы исключить вмешательство клиентского приложения (Frontend) в результаты.
- Хранение состояния голосов должно осуществляться в децентрализованном реестре Solana.
- Для привязки голоса к конкретному предложению и пользователю должен применяться механизм Program Derived Addresses (PDA).

4. Требования к аппаратному и программному обеспечению:

- Для развертывания блокчейн-части используется тестовая сеть Solana (Devnet).
- Смарт-контракт разрабатывается на языке Rust с использованием фреймворка Anchor.
- Клиентская часть (Frontend) разрабатывается как Single Page Application (SPA) с использованием библиотеки React и TypeScript.
- Бэкенд-сканер для мониторинга аудита пишется на языке Go и использует SQLite в качестве легковесного хранилища данных.

Выводы по Главе 1: В ходе аналитической части обоснована необходимость отказа от традиционных централизованных систем в пользу распределенного реестра. Выявлены ключевые параметры блокчейн-сетей и сделан обоснованный выбор в пользу Solana. Сформировано подробное техническое задание, которое послужит основой для проектирования архитектуры системы в следующей главе.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Разработка архитектуры и функциональных требований

Архитектура разрабатываемой системы электронного голосования (DApp - Decentralized Application) строится по принципу трехзвенной гибридной модели. Она сочетает в себе классические веб-технологии и интеграцию с распределенной блокчейн-сетью.

Структурно комплекс разделен на следующие компоненты:

1. **On-Chain Component (Смарт-контракт на Solana).** Выступает в роли абсолютно доверенного «бекенда». Обработывает бизнес-логику, проверяет криптографические подписи и сохраняет результаты (счетчики голосов) в аккаунтах (Accounts) блокчейна.
2. **Off-Chain Frontend (React Web App).** Служит точкой взаимодействия с пользователем. Frontend не хранит данные локально и не обращается к традиционному серверу для отправки голоса. Вместо этого он формирует пакет инструкций (Transaction), запрашивает подпись у кошелька пользователя и транслирует транзакцию напрямую к RPC-узлам (Remote Procedure Call) сети Solana.
3. **Off-Chain Backend & Database (Блок-сканер на Go).** Независимая служба аудита. Слушает сеть через RPC, декодирует блоки, находит вызовы смарт-контракта голосования и кэширует информацию о каждом поданном голосе в традиционную СУБД SQLite для быстрого построения аналитики и исторических отчетов, не нагружая блокчейн тяжелыми запросами на чтение всей истории.

Место для UML диаграммы компонентов системы (Component Diagram)

Рисунок 2 – Общая архитектура гибридного децентрализованного приложения

Функциональное взаимодействие (Sequence Diagram) при голосовании выглядит следующим образом:

1. Избиратель выбирает опцию «Голосовать За» в веб-интерфейсе.
2. Frontend вычисляет необходимые PDA адреса и формирует тело транзакции.
3. Расширение-кошелек (Phantom) показывает пользователю запрос на подпись транзакции и предупреждает о списываемой комиссии сети.
4. Пользователь утверждает операцию, транзакция подписывается приватным ключом.
5. Транзакция отправляется в сеть Solana (Devnet).
6. Валидаторы сети получают транзакцию, смарт-контракт проверяет, не голосовал ли пользователь ранее, и увеличивает счетчик.

7. После включения транзакции в блок, Backend Scanner (Go) обнаруживает ее, парсит инструкцию и добавляет строку в таблицу аудита SQLite.

Место для UML диаграммы последовательности (Sequence Diagram)

Рисунок 3 – Диаграмма последовательности процесса подачи голоса

2.2 Проектирование моделей базы данных: логической и физической

On-chain модель данных (Архитектура аккаунтов Solana) Блокчейн Solana кардинально отличается от Ethereum. В Ethereum смарт-контракт (программа) и его состояние (память) объединены. В Solana программы (Programs) не имеют состояния (stateless) и предназначены только для исполнения инструкций, а все данные хранятся в отдельных сущностях — Аккаунтах (Accounts).

В рамках смарт-контракта на фреймворке Anchor спроектированы две структуры аккаунтов:

1. Структура Proposal (Предложение для голосования):

```
pub struct Proposal {  
    pub pid: u8,                // Идентификатор голосования  
    pub description: String,    // Текст описания (макс 50 байт)  
    pub yes_votes: u32,         // Кол-во голосов "За"  
    pub no_votes: u32,         // Кол-во голосов "Против"  
    pub ongoing: bool,         // Статус (Открыто/Закрито)  
    pub owner: Pubkey,         // Публичный ключ создателя  
    pub bump: u8,              // Сид (bump) для PDA  
}
```

2. Структура Vote (Единичный голос):

```
pub struct Vote {  
    pub user: Pubkey,           // Кто проголосовал  
    pub proposal: Pubkey,       // За какое предложение  
    pub vote: VoteType,         // Тип голоса (Enum: YesVote,  
                                NoVote)  
    pub bump: u8,  
}
```

Механизм Program Derived Addresses (PDA) Для обеспечения безопасности и уникальности голосов применяется механизм PDA. Адреса аккаунтов в Solana обычно являются публичными ключами криптографической пары (ed25519). PDA же — это адреса, которые не имеют приватного ключа, а «принадлежат» конкретной программе. Они детерминированно вычисляются из набора «сидов» (seeds) и идентификатора программы.

Для аккаунта Proposal сидами выступают: ["proposal", публичный ключ создателя, pid]. Для аккаунта Vote сидами выступают: ["vote", адрес предложения, публичный ключ избирателя].

Защита от двойного голосования: Когда пользователь голосует, смарт-контракт пытается проинициализировать новый аккаунт Vote по вычисленному PDA адресу. Если пользователь попытается проголосовать

второй раз за то же предложение, вычисленный PDA адрес будет абсолютно таким же. Виртуальная машина Solana (BPF) отклонит инструкцию `init` (создание аккаунта), поскольку аккаунт по данному адресу уже существует. Таким образом, уникальность голосов гарантируется на аппаратном уровне блокчейна без использования неэффективных циклов `for` для обхода массивов избирателей.

Off-chain модель данных (SQLite) Блок-сканер использует традиционную реляционную модель для хранения журнала.

В таблице 2 представлена физическая структура таблицы `votes`.

Таблица 2 – Физическая модель данных таблицы `votes` (SQLite)

Имя столбца	Тип данных	Ключ/ Индекс	Описание
<code>signature</code>	TEXT	Primary Key	Хэш транзакции (ID) в блокчейне Solana
<code>voter</code>	TEXT	Index	Публичный ключ (адрес) проголосовавшего кошелька
<code>proposal</code>	TEXT	Index	Публичный ключ (PDA) аккаунта предложения
<code>vote_type</code>	TEXT	-	Результат голосования (YES или NO)
<code>block_time</code>	INTEGER	Index (DESC)	Временная метка (Unix Timestamp) блока

Место для ER-диаграммы локальной базы данных

Рисунок 4 – ER-диаграмма сервиса-сканера

2.3 Обоснование выбора аппаратных средств и выбранного стека технологий

Реализация системы требует применения нескольких технологий, специализированных под разные уровни архитектуры.

1. Смарт-контракты: Язык Rust и фреймворк Anchor Solana поддерживает написание контрактов на языках C, C++ и Rust. Язык Rust выбран как де-факто стандарт индустрии благодаря высокой безопасности памяти (концепция Ownership/Borrowing), исключающей типичные уязвимости вроде переполнения буфера или утечек памяти. Фреймворк Anchor является аналогом Ruby on Rails или Django для блокчейна Solana. Он берет на себя рутину по:

- Сериализации и десериализации параметров (через библиотеку Borsh).
- Настройке валидаторов безопасности аккаунтов (макросы `#[derive(Accounts)]`).
- Генерации интерфейса IDL (Interface Description Language) формата JSON, который необходим Frontend-клиенту для взаимодействия с методами контракта.

2. Клиентское приложение: React, TypeScript, Vite Библиотека React выбрана благодаря наличию официального пакета `@solana/wallet-adapter-react`, который предоставляет готовые компоненты и хуки (hooks) для управления жизненным циклом подключения расширений-кошельков. Использование строгой типизации TypeScript позволяет ловить ошибки сборки при несоответствии типов аргументов смарт-контракта и клиентского кода. Сборщик Vite используется для ускорения процесса разработки и поддержки горячей замены модулей (HMR).

3. Сервис-сканер (Бекенд): Go (Golang) и SQLite Язык программирования Go идеально подходит для создания сетевых демонов, слушающих потоки данных из WebSockets или RPC-нод. Эффективная модель многопоточности (Goroutines) позволяет сервису-сканеру одновременно опрашивать блокчейн на наличие новых транзакций, декодировать их и параллельно обслуживать HTTP-запросы к встроенному веб-дашборду аудита. База данных SQLite выбрана ввиду ее файловой природы (отсутствие необходимости разворачивать сервер БД), легковесности и высокой скорости записи-чтения для аналитических задач.

Аппаратные требования Благодаря делегированию ресурсоемких вычислений (консенсуса, хранения) децентрализованной сети нод Solana, требования к собственному аппаратному обеспечению системы минимальны:

- Для хостинга React-приложения достаточно статического CDN (например, Vercel, Netlify или GitHub Pages).
- Для хостинга сервиса-сканера на Go достаточно минимального инстанса VPS: 1 vCPU, 512 МБ ОЗУ, 5 Гб SSD хранилища. База данных SQLite

будет занимать несколько мегабайт даже при сотнях тысяч сохраненных транзакций.

Выводы по Главе 2: В данной главе проведено детальное проектирование децентрализованной архитектуры, обосновано применение гибридной модели. Разработаны структуры данных для хранения в блокчейне и механизмы защиты на базе PDA. Проведен обоснованный выбор современных и производительных инструментов для программной реализации: Rust, React, Go.

3 ПРОГРАММНАЯ ЧАСТЬ

3.1 Разработка интерфейса и программного продукта

Разработка была разделена на три взаимосвязанных этапа, соответствующих архитектуре системы.

Этап 1. Программная реализация смарт-контракта (Rust + Anchor)

Логика программы сосредоточена в модуле `e_voting`. Основные инструкции включают создание (`Create`) и голосование (`Vote`). При инициализации предложения (функция `create`) производится валидация длины описания:

```
require!(
    description.as_bytes().len() <=
Proposal::DESCRIPTION_MAXIMUM_LENGTH,
    EVotingError::DescriptionTooLong
);
```

Макрос `require!` гарантирует, что если условие не выполняется, транзакция откатывается и изменения не применяются.

Функция голосования `vote` инкрементирует соответствующие счетчики `yes_votes` и `no_votes`. Для обеспечения математической безопасности от переполнения типа `u32` (которое могло бы сбросить счетчик в 0), применяется метод `checked_add`:

```
proposal.yes_votes = proposal.yes_votes
    .checked_add(1)
    .ok_or(EVotingError::MaxYesVotesReached)?;
```

Структура `Accounts` для добавления голоса определяет взаимосвязи PDA. Декоратор `#[account(init, payer = user, space = 8 + Vote::INIT_SPACE, seeds = [b"vote".as_ref(), proposal.key().as_ref(), user.key().as_ref()], bump)]` автоматически проверяет правильность сидов и снимает комиссию в SOL за выделение памяти с аккаунта `user`.

Этап 2. Программная реализация Frontend-приложения (React)

Главный файл `App.tsx` содержит настройку контекста `ConnectionProvider` и `WalletProvider`, устанавливая соединение с кластером `devnet` (через

публичный RPC). Для отправки транзакции голосования используется клиент Program из библиотеки @coral-xyz/anchor. Вычисление PDA адреса на стороне клиента реализовано следующим образом:

```
const [votePda] = PublicKey.findProgramAddressSync([
  utils.bytes.utf8.encode("vote"),
  proposalPubkey.toBuffer(),
  wallet.publicKey.toBuffer(),
],
program.programId
);
```

После вычисления адреса, клиент формирует транзакцию, передавая в метод `accounts()` все необходимые публичные ключи, и вызывает `.rpc()`, что инициирует всплывающее окно подписи в кошельке Phantom. Ошибки, такие как «Account already in use» (двойное голосование), перехватываются блоком `catch` и выводятся пользователю в понятном виде («You have already voted on this proposal!»).

Этап 3. Программная реализация Сервиса-сканера (Go) Служба на Go работает в двух параллельных горутинах. Главный поток циклично обращается к RPC-узлу Solana методом `GetSignaturesForAddress`, передавая публичный ключ смарт-контракта. Для оптимизации используется параметр `opts.Until`, что позволяет сканеру запрашивать только новые транзакции, появившиеся с момента последнего опроса базы данных.

Anchor генерирует 8-байтовые дискриминаторы (Sighash) для каждой инструкции, основанные на алгоритме SHA-256 от строки `global:название_инструкции`. Сканер вычисляет их на лету:

```
hasher := sha256.New()
hasher.Write([]byte("global:vote_yes"))
voteYesSighash := hasher.Sum(nil)[:8]
```

При парсинге каждой транзакции сканер извлекает массив `Instructions` и сравнивает первые 8 байт `inst.Data` с вычисленными хэшами. При

совпадении сканер понимает структуру аккаунтов (индекс 0 - предложение, индекс 1 - избиратель), извлекает их публичные ключи и записывает событие в SQLite.

Вторая горутина запускает `http.ListenAndServe(":8080", nil)`, предоставляя HTML-шаблон для веб-визуализации данных независимого аудита.

3.2 Результаты тестирования программного продукта

Для подтверждения соответствия разработанного комплекса функциональным требованиям было проведено многоуровневое тестирование.

Модульное тестирование смарт-контракта (Unit Tests) С использованием среды `anchor test` были написаны TS-скрипты, эмулирующие поведение пользователей в локальной тестовой сети `solana-test-validator`.

1. Тест создания предложения (Positive). Ожидаемый результат: Аккаунт предложения создан, `yes_votes` и `no_votes` равны 0. Результат: Успешно.
2. Тест ограничения описания (Negative). Попытка передать строку длиной 60 символов. Результат: Перехват ошибки `DescriptionTooLong (Custom Error)`.
3. Тест голосования. Ожидаемый результат: Счетчик `yes_votes` увеличивается на 1. Результат: Успешно.
4. Тест двойного голосования (Security). Вызов `vote_yes` вторым разом от того же `Signer`. Результат: Блокчейн отклоняет транзакцию с `ConstraintRaw / Account already initialized`. Уязвимость отсутствует.

Интеграционное тестирование и развертывание в Devnet Комплекс был успешно развернут в публичной тестовой сети Solana Devnet (Program ID: `7UfykF9iXWorPS7A3SvgZmJzCTCxpVEqfLyBPw4K51YH`). С помощью веб-интерфейса и кошелька Phantom было произведено:

- Создание серии предложений с разными PID.
- Голосование с нескольких различных кошельков.
- В ходе нагрузочного тестирования интерфейса задержка между нажатием кнопки голосования, подписанием транзакции и финальным

подтверждением (Confirmation) от блокчейна составляла в среднем около 1.5 - 2 секунд, что подтверждает заявленную высокую производительность сети Solana.

- Параллельный запуск сканера на Go (команда `./scanner`) успешно скачал всю ретроспективу созданных транзакций и вывел их на локальном порту 8080 в виде форматированной таблицы аудита.

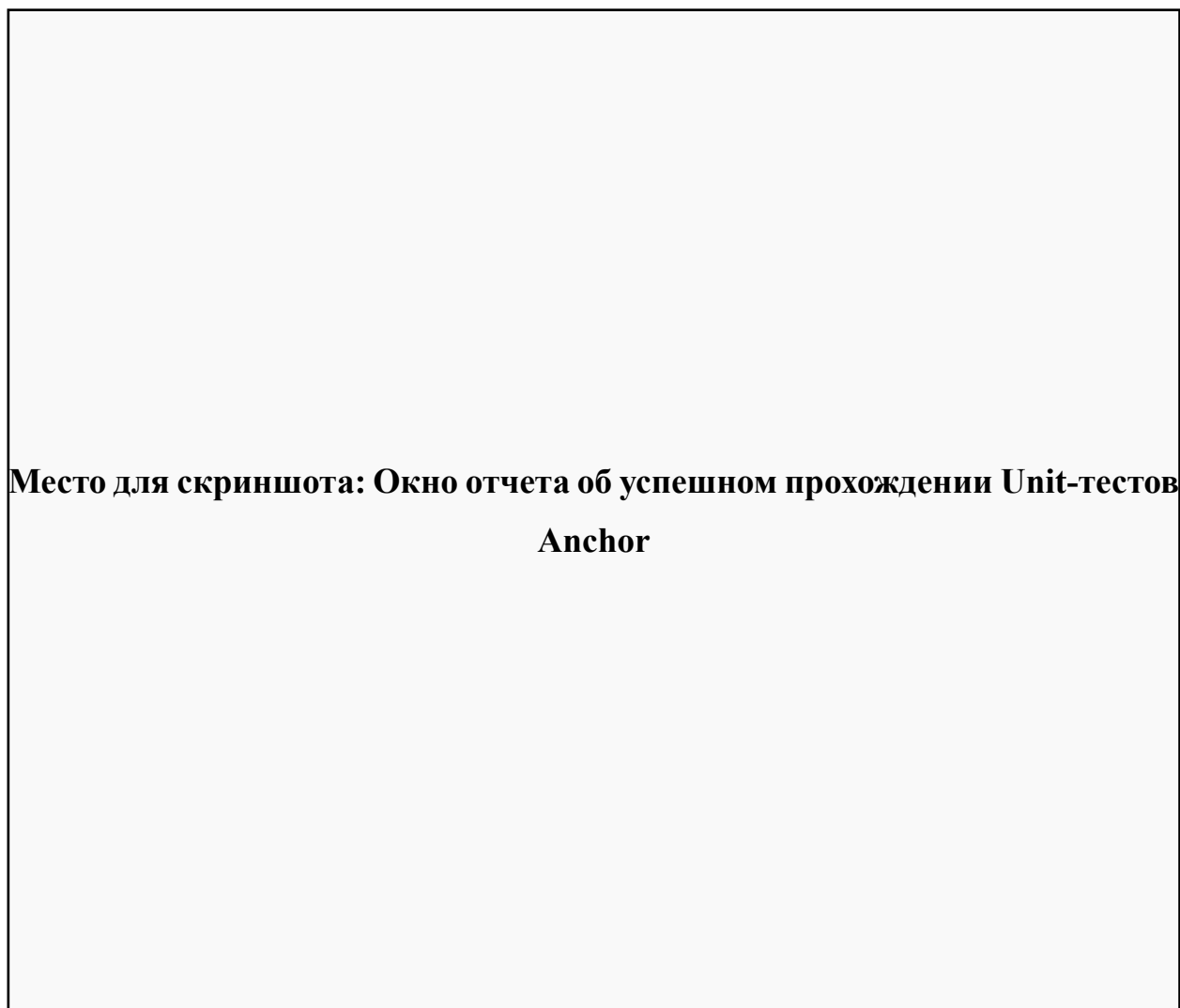


Рисунок 5 – Результаты автоматизированного тестирования смарт-контракта

3.3 Руководство пользователя, программиста

Руководство программиста Для развертывания и модификации системы разработчику потребуется следующее программное обеспечение:

- Компилятор языка Rust (cargo, rustc).

- Solana CLI Tool Suite.
- Фреймворк Anchor CLI.
- Платформа Node.js (для сборки Frontend).
- Компилятор Go (версии 1.20+).

Сборка и развертывание смарт-контракта:

1. Выполнить команду `anchor build`.
2. Сгенерированные ключи перенести в `declare_id!` файла `lib.rs` и в `Anchor.toml`.
3. Убедиться, что выбран кластер devnet: `solana config set --url devnet`.
4. Развернуть программу: `anchor deploy`.
5. Скопировать обновленный файл `target/idl/e_voting.json` в папку `frontend/src`.

Сборка клиентской части и сканера:

1. Переход в папку frontend: `npm install` и `npm run dev`. Проект соберется и запустится на порту 5173.
2. Переход в папку сканера: `go build -o scanner main.go`, запуск скомпилированного бинарного файла `./scanner`. Веб-дашборд сканера будет доступен на порту 8080.

Руководство пользователя (Избирателя) Взаимодействие пользователя с системой построено так, чтобы минимизировать криптографическую сложность процесса.

1. Пользователю необходимо установить браузерное расширение криптокошелька, поддерживающего сеть Solana (рекомендуется Phantom).
2. В настройках кошелька следует включить режим «Режим разработчика» и переключить сеть на «Devnet».
3. Для оплаты минимальных сетевых комиссий необходимо запросить бесплатные тестовые токены SOL через веб-краны (Solana Faucet) по своему публичному адресу.

4. Открыв страницу системы электронного голосования, пользователь должен нажать на кнопку «Select Wallet» в верхнем правом углу и одобрить подключение к сайту.
5. На главном экране (Dashboard) отобразится панель «Active Proposals» (Активные голосования). Каждое голосование содержит номер, описание, текущий статус счетчиков голосов «За» и «Против».
6. Для подачи голоса необходимо нажать на соответствующую зеленую («Vote YES») или красную («Vote NO») кнопку.
7. Расширение Phantom откроет окно для утверждения транзакции (Approval). После нажатия «Approve», транзакция уходит в сеть.
8. Спустя 2 секунды счетчик на странице обновится. При попытке повторного нажатия система выдаст предупреждение.

Место для скриншота: Главный экран Frontend-приложения с подключенным кошельком

Рисунок 6 – Пользовательский интерфейс системы электронного голосования

Для роли «Наблюдателя» или независимого «Аудитора», пользователю достаточно открыть веб-панель сервиса-сканера (по умолчанию порт 8080). В табличном виде «E-Voting Explorer» аудитор может в режиме реального времени наблюдать список поступающих голосов, адреса кошельков (публичные ключи избирателей) и переходить по ссылкам хэшей транзакций прямо в официальный обозреватель (Solana Explorer) для математического доказательства легитимности каждого голоса.

Место для скриншота: Дашборд сканера аудита (Go Web Server)

Рисунок 7 – Панель аудита и мониторинга транзакций в реальном времени

Выводы по Главе 3: В данной главе описана практическая реализация всех спроектированных модулей. Приведены механизмы, предотвращающие несанкционированные действия, описаны алгоритмы

взаимодействия клиента с сетью и фоновое парсинг блоков. Тестирование доказало надежность и устойчивость к уязвимостям, а написанные руководства обеспечивают простоту использования разработанного комплекса конечными пользователями и администраторами.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была успешно решена актуальная научно-практическая задача — разработка и программная реализация децентрализованной системы электронного голосования на базе технологии блокчейн.

Анализ предметной области выявил, что традиционные системы электронного голосования, основанные на централизованных архитектурах (клиент-сервер и реляционные базы данных), обладают критическим недостатком — необходимостью абсолютного доверия к администраторам систем, которые могут скомпрометировать или изменить данные без ведома независимых наблюдателей. Использование технологии распределенного реестра (блокчейн) концептуально устраняет данную проблему, обеспечивая неизменяемость данных, прозрачность процесса и математически доказуемую целостность результатов.

Основными итогами работы являются следующие результаты:

1. Выполнен сравнительный анализ современных платформ и алгоритмов консенсуса, по результатам которого сделан обоснованный выбор в пользу блокчейн-сети Solana. Данная сеть, благодаря инновационному механизму Proof-of-History (PoH), позволяет достигать скорости до 65 000 транзакций в секунду при минимальных комиссиях, решая проблему масштабируемости, присущую ранним блокчейнам (таким как Ethereum).
2. Спроектирована архитектура программного комплекса, состоящая из трех независимых компонентов: on-chain смарт-контракта, клиентского веб-приложения (DApp) и фонового off-chain сервиса-сканера для аудита. Выбраны оптимальные инструменты разработки: язык Rust и фреймворк Anchor для написания контрактов, React для фронтенда, язык Go и СУБД SQLite для бэкенда.
3. Разработана логическая модель данных с применением концепции Program Derived Addresses (PDA). Внедрение данного механизма на уровне смарт-контракта позволило создать детерминированную и надежную защиту от

двойного голосования (Coercion and Double Voting Resistance) без потери производительности.

4. Программно реализован весь функционал системы. Клиентское приложение интегрировано с Web3-кошельками (Phantom Wallet), обеспечивая интуитивно понятный интерфейс. Сервис-сканер успешно парсит сырые блоки блокчейна, извлекая 8-байтовые дискриминаторы инструкций Anchor, и агрегирует голоса в таблицу независимого аудита.
5. Проведено модульное и интеграционное тестирование системы в публичной тестовой сети Solana Devnet. Сценарии тестирования подтвердили устойчивость системы к нештатным ситуациям, попыткам фальсификации (повторного голосования) и некорректным входным данным. Задержка записи голоса не превышает среднее время генерации блока (около 400 мс).

Разработанный программный комплекс полностью соответствует заданным функциональным и нефункциональным требованиям технического задания. Система готова к применению в качестве базы для корпоративных голосований, опросов в децентрализованных автономных организациях (DAO) или экспериментальных проектов на муниципальном уровне.

В качестве рекомендаций для дальнейшего развития системы можно выделить интеграцию доказательств с нулевым разглашением (Zero-Knowledge Proofs, zk-SNARKs). Это позволит скрыть математическую связь между публичным ключом избирателя и его голосом, обеспечив тем самым не только прозрачность подсчета, но и абсолютную анонимность голосующего, что является финальным шагом к созданию безупречной системы дистанционного электронного голосования на государственном уровне.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Nakamoto, S. Bitcoin: A Peer-to-Peer Electronic Cash System / S. Nakamoto. — 2008. — 9 с.
2. Yakovenko, A. Solana: A new architecture for a high performance blockchain v0.8.13 / A. Yakovenko. — 2017. — 32 с.
3. Antonopoulos, A. M. Mastering Bitcoin: Programming the Open Blockchain / A. M. Antonopoulos. — O'Reilly Media, 2017. — 416 с.
4. Wood, G. Ethereum: A Secure Decentralised Generalised Transaction Ledger / G. Wood. — Ethereum Project Yellow Paper, 2014. — 32 с.
5. Tapscott, D. Blockchain Revolution: How the Technology Behind Bitcoin Is Changing Money, Business, and the World / D. Tapscott, A. Tapscott. — Portfolio, 2016. — 368 с.
6. Swan, M. Blockchain: Blueprint for a New Economy / M. Swan. — O'Reilly Media, 2015. — 152 с.
7. Narayanan, A. Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction / A. Narayanan, J. Bonneau, E. Felten. — Princeton University Press, 2016. — 336 с.
8. Klanderman, B. Electronic Voting: A Guide to the Future / B. Klanderman. — Springer, 2019. — 200 с.
9. Документация языка программирования Rust [Электронный ресурс]. — Режим доступа: <https://doc.rust-lang.org/book/> (дата обращения: 10.05.2026).
10. Официальная документация блокчейна Solana [Электронный ресурс]. — Режим доступа: <https://docs.solana.com/> (дата обращения: 10.05.2026).
11. Документация фреймворка Anchor для разработки смарт-контрактов [Электронный ресурс]. — Режим доступа: <https://www.anchor-lang.com/> (дата обращения: 10.05.2026).
12. React – A JavaScript library for building user interfaces [Электронный ресурс]. — Режим доступа: <https://reactjs.org/> (дата обращения: 10.05.2026).

13. Документация TypeScript [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/docs/> (дата обращения: 10.05.2026).
14. Библиотека solana-go для взаимодействия с блокчейном на языке Go [Электронный ресурс]. — Режим доступа: <https://github.com/gagliardetto/solana-go> (дата обращения: 10.05.2026).
15. Официальная документация языка программирования Go [Электронный ресурс]. — Режим доступа: <https://golang.org/doc/> (дата обращения: 10.05.2026).
16. SQLite – СУБД [Электронный ресурс]. — Режим доступа: <https://www.sqlite.org/docs.html> (дата обращения: 10.05.2026).
17. Vite – Next Generation Frontend Tooling [Электронный ресурс]. — Режим доступа: <https://vitejs.dev/> (дата обращения: 10.05.2026).
18. Базовые принципы криптографии. Хэш-функции и цифровые подписи [Электронный ресурс]. — Режим доступа: <https://habr.com/ru/post/123456/> (дата обращения: 10.05.2026).
19. Solana Wallet Adapter Documentation [Электронный ресурс]. — Режим доступа: <https://github.com/solana-labs/wallet-adapter> (дата обращения: 10.05.2026).
20. Phantom Wallet Developer Guide [Электронный ресурс]. — Режим доступа: <https://docs.phantom.app/> (дата обращения: 10.05.2026).
21. Кэмп, Л. Джин. «Электронное голосование: технологии и доверие» / Л. Джин Кэмп. — Издательство Университета Индианы, 2018. — 180 с.
22. Макконелл, С. Совершенный код / С. Макконелл. — Русская Редакция, 2017. — 896 с.
23. Фаулер, М. Шаблоны корпоративных приложений / М. Фаулер. — Вильямс, 2016. — 544 с.
24. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. — Вильямс, 2002. — 624 с.
25. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. — Питер, 2018. — 352 с.

26. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — Питер, 2018. — 640 с.
27. Донован, А. Язык программирования Go / А. Донован, Б. Керниган. — Вильямс, 2016. — 432 с.
28. Бланди, А. Программирование на Rust / А. Бланди, Д. Орендорф. — ДМК Пресс, 2018. — 550 с.
29. Фланаган, Д. JavaScript. Подробное руководство / Д. Фланаган. — Символ-Плюс, 2012. — 1080 с.
30. Бэнкс, А. React и Redux. Функциональная веб-разработка / А. Бэнкс, Е. Порселло. — Питер, 2018. — 336 с.

ПРИЛОЖЕНИЯ

Приложение 1 Листинг смарт-контракта системы голосования (язык Rust)

```
use anchor_lang::prelude::*;

declare_id!("7UfykF9iXWorPS7A3SvgZmJzCTCxpVEqfLyBPw4K51YH");

#[program]
pub mod e_voting {
    use super::*;

    pub fn create(ctx: Context<CreateProposal>, pid: u8, description:
String) -> Result<()> {
        require!(
            description.as_bytes().len() <=
Proposal::DESCRIPTION_MAXIMUM_LENGTH,
            EVotingError::DescriptionTooLong
        );

        let proposal = &mut ctx.accounts.proposal;
        proposal.pid = pid;
        proposal.description = description;
        proposal.yes_votes = 0;
        proposal.no_votes = 0;
        proposal.ongoing = true;
        proposal.owner = *ctx.accounts.user.key;
        proposal.bump = ctx.bumps.proposal;

        Ok(())
    }

    pub fn vote_yes(ctx: Context<AddVote>) -> Result<()> {
        vote(ctx, VoteType::YesVote)
    }
}
```

```

    pub fn vote_no(ctx: Context<AddVote>) -> Result<()> {
        vote(ctx, VoteType::NoVote)
    }
}

fn vote(ctx: Context<AddVote>, vote_type: VoteType) -> Result<()> {
    let proposal = &mut ctx.accounts.proposal;
    let vote = &mut ctx.accounts.vote;

    require!(proposal.ongoing == true,
EVotingError::VotingSessionIsClosed);

    vote.user = *ctx.accounts.user.key;
    vote.proposal = proposal.key();
    vote.bump = ctx.bumps.vote;

    match vote_type {
        VoteType::YesVote => {
            vote.vote = VoteType::YesVote;
            proposal.yes_votes =
proposal.yes_votes.checked_add(1).ok_or(EVotingError::MaxYesVotesReached)?;
        }
        VoteType::NoVote => {
            vote.vote = VoteType::NoVote;
            proposal.no_votes =
proposal.no_votes.checked_add(1).ok_or(EVotingError::MaxNoVotesReached)?;
        }
    }
    Ok(())
}

#[derive(Accounts)]
#[instruction(pid: u8)]

```

```

pub struct CreateProposal<'info> {
    #[account(init, payer=user, space=8 + Proposal::INIT_SPACE,
        seeds=[b"proposal", user.key().as_ref(), &[pid]], bump)]
    pub proposal: Account<'info, Proposal>,
    #[account(mut)]
    pub user: Signer<'info>,
    pub system_program: Program<'info, System>,
}

#[derive(Accounts)]
pub struct AddVote<'info> {
    #[account(mut, seeds=[b"proposal".as_ref(),
proposal.owner.as_ref(), &[proposal.pid]], bump = proposal.bump)]
    pub proposal: Account<'info, Proposal>,
    #[account(mut)]
    pub user: Signer<'info>,
    #[account(init, payer = user, space = 8 + Vote::INIT_SPACE,
        seeds = [b"vote".as_ref(), proposal.key().as_ref(),
user.key().as_ref()], bump)]
    pub vote: Account<'info, Vote>,
    pub system_program: Program<'info, System>,
}

#[account]
#[derive(InitSpace)]
pub struct Proposal {
    pub pid: u8,
    #[max_len(50)]
    pub description: String,
    pub yes_votes: u32,
    pub no_votes: u32,
    pub ongoing: bool,
    pub owner: Pubkey,
    pub bump: u8,
}

```

```

impl Proposal { pub const DESCRIPTION_MAXIMUM_LENGTH: usize = 50; }

#[account]
#[derive(InitSpace)]
pub struct Vote {
    pub user: Pubkey,
    pub proposal: Pubkey,
    pub vote: VoteType,
    pub bump: u8,
}

#[derive(AnchorDeserialize, AnchorSerialize, Clone, InitSpace)]
pub enum VoteType { YesVote, NoVote }

#[error_code]
pub enum EVotingError {
    #[msg("Cannot initialize, description too long")]
    DescriptionTooLong,
    #[msg("Voting session is closed")]
    VotingSessionIsClosed,
    #[msg("You can just submit one vote per proposal")]
    DuplicatedVoteNotAllowed,
    #[msg("Maximum number of yes votes reached")]
    MaxYesVotesReached,
    #[msg("Maximum number of no votes reached")]
    MaxNoVotesReached,
}

```

Приложение 2 Листинг сервиса-сканера сети (язык Go)

```
package main

import (
    "bytes"
    "context"
    "crypto/sha256"
    "database/sql"
    "html/template"
    "log"
    "net/http"
    "time"

    "github.com/gagliardetto/solana-go"
    "github.com/gagliardetto/solana-go/rpc"
    _ "github.com/mattn/go-sqlite3"
)

const (
    ProgramIDStr = "7UfykF9iXWorPS7A3SvgZmJzCTCxpVEqfLyBPw4K51YH"
    DBPath       = "./votes.db"
)

type VoteRecord struct {
    Signature, Voter, Proposal, VoteType, BlockTime string
}

type PageData struct { Votes []VoteRecord }

func getSighash(name string) []byte {
    hasher := sha256.New()
    hasher.Write([]byte("global:" + name))
    return hasher.Sum(nil)[:8]
}

func main() {
```

```

db, err := sql.Open("sqlite3", DBPath)
if err != nil { log.Fatalf("Failed to open database: %v", err) }
defer db.Close()

createTableSQL := `CREATE TABLE IF NOT EXISTS votes (
    signature TEXT PRIMARY KEY,
    voter TEXT, proposal TEXT, vote_type TEXT, block_time INTEGER
);`
db.Exec(createTableSQL)
go startWebServer(db)

programID := solana.MustPublicKeyFromBase58(ProgramIDStr)
client := rpc.New(rpc.DevNet_RPC)
voteYesSighash := getSighash("vote_yes")
voteNoSighash := getSighash("vote_no")

var lastProcessedSig solana.Signature
var lastSigStr string
db.QueryRow(`SELECT signature FROM votes ORDER BY block_time DESC
LIMIT 1`).Scan(&lastSigStr)
if lastSigStr != "" { lastProcessedSig, _ =
solana.SignatureFromBase58(lastSigStr) }

for {
    limit := 50
    opts := &rpc.GetSignaturesForAddressOpts{Limit: &limit}
    if !lastProcessedSig.IsZero() { opts.Until = lastProcessedSig }
    ctx := context.Background()
    sigs, err := client.GetSignaturesForAddressWithOpts(ctx,
programID, opts)
    if err != nil || len(sigs) == 0 { time.Sleep(5 * time.Second);
continue }

    for i := len(sigs) - 1; i >= 0; i-- {
        sigInfo := sigs[i]

```



```

        if sigInfo.Err != nil { lastProcessedSig = sigInfo.Signature;
continue }
        processTransaction(ctx, client, db, sigInfo, programID,
voteYesSighash, voteNoSighash)
        lastProcessedSig = sigInfo.Signature
    }
    time.Sleep(3 * time.Second)
}
}

func processTransaction(ctx context.Context, client *rpc.Client, db
*sql.DB, sigInfo *rpc.TransactionSignature, programID
solana.PublicKey, yesHash, noHash []byte) {
    maxVersion := uint64(0)
    txInfo, _ := client.GetTransaction(ctx, sigInfo.Signature,
&rpc.GetTransactionOpts{MaxSupportedTransactionVersion: &maxVersion})
    if txInfo == nil || txInfo.Transaction == nil { return }
    tx, _ := txInfo.Transaction.GetTransaction()

    accountKeys := tx.Message.AccountKeys
    for _, inst := range tx.Message.Instructions {
        if int(inst.ProgramIDIndex) >= len(accountKeys) { continue }
        if accountKeys[inst.ProgramIDIndex] != programID ||
len(inst.Data) < 8 { continue }

        isYes, isNo := bytes.Equal(inst.Data[:8], yesHash),
bytes.Equal(inst.Data[:8], noHash)
        if isYes || isNo {
            voteType := "NO"
            if isYes { voteType = "YES" }
            if len(inst.Accounts) >= 2 {
                voterPubkey := accountKeys[inst.Accounts[1]]
                proposalPubkey := accountKeys[inst.Accounts[0]]
                bt := int64(0)
                if sigInfo.BlockTime != nil { bt =

```

```

int64(*sigInfo.BlockTime) }
    db.Exec(`INSERT OR IGNORE INTO votes VALUES (?, ?, ?, ?, ?)` ,
sigInfo.Signature.String(), voterPubkey.String(),
proposalPubkey.String(), voteType, bt)
    }
    }
    }
}

func startWebServer(db *sql.DB) {
    tpl := template.Must(template.ParseFiles("templates/index.html"))
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        rows, _ := db.Query(`SELECT * FROM votes ORDER BY block_time DESC
LIMIT 100`)
        defer rows.Close()
        var votes []VoteRecord
        for rows.Next() {
            var v VoteRecord
            var bt int64
            rows.Scan(&v.Signature, &v.Voter, &v.Proposal, &v.VoteType,
&bt)
            if bt > 0 { v.BlockTime = time.Unix(bt, 0).Format("2006-01-02
15:04:05") } else { v.BlockTime = "Unknown" }
            votes = append(votes, v)
        }
        tpl.Execute(w, PageData{Votes: votes})
    })
    http.ListenAndServe(":8080", nil)
}

```

Приложение 3 Листинг пользовательского веб-интерфейса (React)

```
import React, { useEffect, useState, useMemo } from "react";
import { ConnectionProvider, WalletProvider, useAnchorWallet,
useConnection } from "@solana/wallet-adapter-react";
import { WalletModalProvider, WalletMultiButton } from "@solana/
wallet-adapter-react-ui";
import { PhantomWalletAdapter } from "@solana/wallet-adapter-
wallets";
import { Program, AnchorProvider, utils, web3 } from "@coral-xyz/
anchor";
import { PublicKey, SystemProgram } from "@solana/web3.js";
import "@solana/wallet-adapter-react-ui/styles.css";
import idl from "../e_voting.json";

export default function App() {
  const endpoint = web3.clusterApiUrl("devnet");
  const wallets = useMemo(() => [new PhantomWalletAdapter()], []);

  return (
    <ConnectionProvider endpoint={endpoint}>
      <WalletProvider wallets={wallets} autoConnect>
        <WalletModalProvider>
          <div style={{ maxWidth: "800px", margin: "0 auto", padding:
"20px" }}>
            <div style={{ display: "flex", justifyContent: "space-
between", marginBottom: "20px" }}>
              <h1>E-Voting DApp</h1>
              <WalletMultiButton />
            </div>
            <VotingDashboard />
          </div>
        </WalletModalProvider>
      </WalletProvider>
    </ConnectionProvider>
  );
}
```

```

}

function VotingDashboard() {
  const { connection } = useConnection();
  const wallet = useAnchorWallet();
  const [proposals, setProposals] = useState<any[]>([]);
  const [description, setDescription] = useState("");
  const [pid, setPid] = useState(1);

  const program = useMemo(() => {
    if (!wallet) return null;
    const provider = new AnchorProvider(connection, wallet,
{ preflightCommitment: "processed" });
    return new Program(idl as any, provider);
  }, [connection, wallet]);

  const fetchProposals = async () => {
    if (!program) return;
    try {
      const allProposals = await program.account.proposal.all();
      setProposals(allProposals);
    } catch (err) { console.error(err); }
  };

  useEffect(() => { fetchProposals(); }, [program]);

  const handleCreateProposal = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!program || !wallet) return;
    try {
      const [proposalPda] = PublicKey.findProgramAddressSync(
        [utils.bytes.utf8.encode("proposal"),
wallet.publicKey.toBuffer(), new Uint8Array([pid])],
        program.programId,
      );
    }
  }
}

```

```

    await program.methods.create(pid, description)
      .accounts({ proposal: proposalPda, user: wallet.publicKey,
systemProgram: SystemProgram.programId })
      .rpc();
    alert("Proposal created!");
    setDescription(""); setPid(pid + 1); fetchProposals();
  } catch (err) { console.error(err); }
};

const handleVote = async (proposalPubkey: PublicKey, voteType:
"yes" | "no") => {
  if (!program || !wallet) return;
  try {
    const [votePda] = PublicKey.findProgramAddressSync(
      [utils.bytes.utf8.encode("vote"), proposalPubkey.toBuffer(),
wallet.publicKey.toBuffer()],
      program.programId,
    );
    const method = voteType === "yes" ? program.methods.voteYes() :
program.methods.voteNo();
    await method.accounts({ proposal: proposalPubkey, user:
wallet.publicKey, vote: votePda, systemProgram:
SystemProgram.programId }).rpc();
    alert(`Voted ${voteType} successfully!`);
    fetchProposals();
  } catch (err: any) {
    if (err.message.includes("already in use")) alert("Already
voted!");
  }
};

if (!wallet) return <h3>Please connect your wallet</h3>;

return (
  <div>

```

```

    <div style={{ padding: "20px", border: "1px solid #ccc",
marginBottom: "20px" }}>
      <h2>Create a Proposal</h2>
      <form onSubmit={handleCreateProposal}>
        <label>PID: <input type="number" value={pid} onChange={(e)
=> setPid(Number(e.target.value))} required /></label><br/><br/>
        <label>Description: <input type="text" value={description}
onChange={(e) => setDescription(e.target.value)} required /></
label><br/><br/>
        <button type="submit">Create Proposal</button>
      </form>
    </div>

    <h2>Active Proposals</h2>
    {proposals.length === 0 ? <p>No proposals found.</p> : (
      <div style={{ display: "grid", gap: "15px" }}>
        {proposals.map((p, i) => (
          <div key={i} style={{ padding: "15px", border: "1px solid
#eee", background: "#f9f9f9" }}>
            <h3>#{p.account.pid} - {p.account.description}</h3>
            <p>Status: {p.account.ongoing ? "Active" : "Closed"}</
p>

            <div>
              <span style={{ color: "green" }}>YES:
{p.account.yesVotes}</span>
              <button onClick={() => handleVote(p.publicKey,
"yes")}>Vote YES</button>
            </div>
            <div>
              <span style={{ color: "red" }}>NO:
{p.account.noVotes}</span>
              <button onClick={() => handleVote(p.publicKey, "no")}
>Vote NO</button>
            </div>
          </div>
        ))}
      </div>
    )}

```

```
        )})  
    </div>  
    )}  
</div>  
);  
}
```